



Date:

Course Code:23DS6PCNGD

Course:Next Gen Databases

USN No.:

Program 1:PostgreSQL Query Questions - Hospital Management System

Create a database for a hospital management system with tables: Patients (PatientID, Name, Age, Gender, AdmissionDate) and Doctors (DoctorID, Name, Specialization, Experience).

- Fetch Patient Details:
- List Patients Admitted on a Specific Date ('2024-06-15'):
- List Patients by Age Group:
- Update Patient Details (Change Name for PatientID = 1):
- List Doctors by Specialization ('Cardiology'):
- Count Patients per Doctor:
- Calculate Average Experience of Doctors.

Query and Solutions:

1. Create Database

```
CREATE DATABASE hospital_db;  
\c hospital_db;
```

2. Create Tables

```
CREATE TABLE Patients (  
    PatientID SERIAL PRIMARY KEY,  
    Name VARCHAR(100),  
    Age INT,  
    Gender VARCHAR(10),  
    AdmissionDate DATE,  
    DoctorID INT  
);
```

```
CREATE TABLE Doctors  
CREATE TABLE Doctors (  
    DoctorID SERIAL PRIMARY KEY,  
    Name VARCHAR(100),  
    Specialization VARCHAR(100),  
    Experience INT -- in years  
);
```

3. Sample Data Insertion

```
INSERT INTO Doctors (Name, Specialization, Experience) VALUES  
('Dr. Smith', 'Cardiology', 15),  
('Dr. Gupta', 'Neurology', 10),  
('Dr. Allen', 'Orthopedics', 8);
```

```
INSERT INTO Patients (Name, Age, Gender, AdmissionDate, DoctorID) VALUES  
('Alice', 30, 'Female', '2024-06-15', 1),  
('Bob', 45, 'Male', '2024-06-16', 2),
```



Date:

Course Code:23DS6PCNGD

Course:Next Gen Databases

USN No.:

('Clara', 65, 'Female', '2024-06-15', 1),
('David', 23, 'Male', '2024-06-17', 3);

Query Solutions

a) Fetch All Patient Details

```
SELECT * FROM Patients;
```

b) List Patients Admitted on a Specific Date ('2024-06-15')

```
SELECT * FROM Patients  
WHERE AdmissionDate = '2024-06-15';
```

c) List Patients by Age Group (Example: 30-50 years)

```
SELECT CASE  
    WHEN Age < 18 THEN 'Child'  
    WHEN Age BETWEEN 18 AND 40 THEN 'Young Adult'  
    WHEN Age BETWEEN 41 AND 60 THEN 'Middle Aged'  
    ELSE 'Senior'  
END AS AgeGroup,  
COUNT(*) AS TotalPatients  
FROM Patients  
GROUP BY AgeGroup;
```

d) Update Patient Details (Change Name for PatientID = 1)

```
UPDATE Patients  
SET Name = 'Alicia'  
WHERE PatientID = 1;
```

e) List Doctors by Specialization ('Cardiology')

```
SELECT * FROM Doctors  
WHERE Specialization = 'Cardiology';
```

f) Count Patients Per Doctor

```
SELECT d.Name AS DoctorName, COUNT(p.PatientID) AS TotalPatients  
FROM Doctors d  
LEFT JOIN Patients p ON d.DoctorID = p.DoctorID  
GROUP BY d.Name;
```

g) Calculate Average Experience of Doctors

```
SELECT AVG(Experience) AS AvgExperience  
FROM Doctors;
```



Date:

Course Code:23DS6PCNGD

Course:Next Gen Databases

USN No.:

2.CRUD operations in CouchDB Student Database Create a student database with the fields: (SRN, Sname, Degree, Sem, CGPA) a) Display all the documents b) Display all the students in BCA c) Display all the students in ascending order d) Display first 5 students e) Display students 5,6,7 f) List the degree of student "Rahul" g) Display students details of 5,6,7 in descending order of percentage h) Display all the BCA students with CGPA greater than 6, but less than 7.5

Suppose you created a student database with documents like:json

```
{
  "_id": "student1",
  "SRN": "001",
  "Sname": "Rahul",
  "Degree": "BCA",
  "Sem": 5,
  "CGPA": 7.2
}
```

(similar documents for others)

a) Display all documents

REST API:

GET /student/_all_docs?include_docs=true

This shows all student documents with full details.

b) Display all students in BCA

Create a View:

Map function (design document):

```
function (doc) {
  if (doc.Degree === 'BCA') {
    emit(doc._id, doc);
  }
}
```

Query the view:



Date:

Course Code:23DS6PCNGD

Course:Next Gen Databases

USN No.:

GET /student/_design/degree/_view/bca_students

c) Display all students in ascending order

Create a View where you emit name as key:

```
function (doc) {  
    emit(doc.Sname, doc);  
}
```

By default, CouchDB returns in ascending order of keys.

Query:

GET /student/_design/sort/_view/by_name

d) Display first 5 students

Use limit parameter:

GET /student/_design/sort/_view/by_name?limit=5

e) Display students 5,6,7

Use skip and limit:

GET /student/_design/sort/_view/by_name?skip=4&limit=3

- skip=4 → Skip first 4 students (start from 5th)
- limit=3 → Get 3 students (5,6,7)

f) List the degree of student "Rahul"

Option 1: If you know Rahul's document ID, directly:

GET /student/student1

and read the Degree field.

Option 2: Create a view to find by name:

```
function (doc) {  
    if (doc.Sname === 'Rahul') {  
        emit(doc._id, doc.Degree);  
    }  
}
```

Query:

GET /student/_design/degree/_view/by_name



Date:

Course Code:23DS6PCNGD

Course:Next Gen Databases

USN No.:

g) Display students 5,6,7 in descending order of CGPA

Create a view where you emit CGPA as key:

```
function (doc) {  
    emit(doc.CGPA, doc);  
}
```

Query:

GET /student/_design/sort/_view/by_cgpa?descending=true&skip=4&limit=3

- descending=true for descending CGPA.
- skip=4, limit=3 to get students 5,6,7 in descending order.

h) Display all BCA students with CGPA > 6 and < 7.5

More advanced map:

```
function (doc) {  
    if (doc.Degree === 'BCA' && doc.CGPA > 6 && doc.CGPA < 7.5) {  
        emit(doc._id, doc);  
    }  
}
```

Query:

GET /student/_design/degree/_view/bca_cgpa



Date:

Course Code:23DS6PCNGD

Course:Next Gen Databases

USN No.:

3.CRUD operations in CouchDB Library Database

Create a library database with the fields: (ISBN, Title, Author, Genre, PublicationYear, CopiesAvailable, Rating).

- a) **Display all the documents in the library database.**
- b) **Display all the books in the genre "Fiction".**
- c) **Display all books sorted by their Title in alphabetical order.**
- d) **Display the first 3 books.**
- e) **Display books 4, 5, and 6.**
- f) **List the Author of the book titled "The Great Gatsby".**
- d) **Operations for library Database**

a) Display all the documents

REST API (cURL command):

`curl -X GET http://admin:1234@localhost:5984/library/_all_docs?include_docs=true`

- `include_docs=true` fetches full documents.

b) Display all the books in the genre "Fiction"

First, create a **View** in a design document.

Map Function:

```
function (doc) {  
  if (doc.Genre === "Fiction") {  
    emit(doc._id, doc);  
  }  
}
```

Query:

`curl -X GET http://admin:1234@localhost:5984/library/_design/genre/_view/fiction_books`

c) Display all books sorted by their Title alphabetically



Date:

Course Code:23DS6PCNGD

Course:Next Gen Databases

USN No.:

Create another **View**.

Map Function:

```
function (doc) {  
    emit(doc.Title, doc);  
}
```

(save inside `_design/sort`)

Query:

```
curl -X GET http://admin:1234@localhost:5984/library/_design/sort/_view/by_title
```

- By default, sorted **ascending** by Title.

d) Display the first 3 books

Use the above view and limit results:

```
curl -X GET "http://admin:1234@localhost:5984/library/_design/sort/_view/by_title?limit=3"
```

e) Display books 4, 5, and 6

Use skip and limit:

```
curl -X GET  
"http://admin:1234@localhost:5984/library/_design/sort/_view/by_title?skip=3&limit=3"
```

- `skip=3`: Skip the first 3 books.
- `limit=3`: Get next 3 books (i.e., 4th, 5th, and 6th).

f) List the Author of the book titled "The Great Gatsby"

Create a View emitting Title as key and Author as value:

Map Function:

```
function (doc) {  
    emit(doc.Title, doc.Author);  
}
```

(saved inside `_design/title_author`)

Then query the view with a specific key:



Date:

Course Code:23DS6PCNGD

Course:Next Gen Databases

USN No.:

```
curl -X GET
```

```
"http://admin:1234@localhost:5984/library/_design/title_author/_view/by_title?key=\"The Great Gatsby\""
```

4.CRUD Operations in Neo4j - Social Network Database

Create a social network database with nodes: User (UserID, Username) and relationships: (FOLLOWS).

- Write a query to display all users.
- Write a query to display users followed by a specific user with Username "Jane".
- Write a query to display all users in ascending order by Username.
- Write a query to find users who follow both "Alice" and "Bob".
- Write a query to find users with the most number of followers.
- Display first 5 users:

// Creating Users

```
CREATE (u1:User {UserID: 1, Username: 'Alice'})
```

```
(u2:User {UserID: 2, Username: 'Bob'})
```

```
(u3:User {UserID: 3, Username: 'Charlie'})
```

```
(u4:User {UserID: 4, Username: 'David'})
```

```
(u5:User {UserID: 5, Username: 'Eve'})
```

```
(u6:User {UserID: 6, Username: 'Frank'})
```

```
(u7:User {UserID: 7, Username: 'Grace'})
```

```
(u8:User {UserID: 8, Username: 'Hannah'})
```

```
(u9:User {UserID: 9, Username: 'Jane'})
```

```
(u10:User {UserID: 10, Username: 'Ivy'})
```

// Creating Follows Relationships

```
CREATE (u1)-[:FOLLOWS]->(u2)
```

```
(u1)-[:FOLLOWS]->(u3)
```




Date:

Course Code:23DS6PCNGD

Course:Next Gen Databases

USN No.:

(u2)-[:FOLLOWS]->(u4)

(u3)-[:FOLLOWS]->(u1)

(u4)-[:FOLLOWS]->(u5)

(u5)-[:FOLLOWS]->(u6)

(u6)-[:FOLLOWS]->(u7)

CREATE (u7)-[:FOLLOWS]->(u8)

CREATE (u8)-[:FOLLOWS]->(u9)

CREATE (u9)-[:FOLLOWS]->(u10)

a) Display all users

MATCH (u:User)

RETURN u.Username AS Username, u.UserID AS UserID

b) Display users followed by a specific user with Username "Jane"

MATCH (jane:User {Username: 'Jane'})-[:FOLLOWS]->(followers)

RETURN followers.Username AS FollowedUser

c) Display all users in ascending order by Username

MATCH (u:User)

RETURN u.Username AS Username, u.UserID AS UserID

ORDER BY u.Username ASC

d) Find users who follow both "Alice" and "Bob"

MATCH (u:User)-[:FOLLOWS]->(a:User {Username: 'Alice'})

MATCH (u)-[:FOLLOWS]->(b:User {Username: 'Bob'})

RETURN u.Username AS UserFollowingBoth

e) Find users with the most number of followers



Date:

Course Code:23DS6PCNGD

Course:Next Gen Databases

USN No.:

MATCH (follower:User)-[:FOLLOWS]->(u:User)

WITH u, COUNT(follower) AS numFollowers

ORDER BY numFollowers DESC

LIMIT 1

RETURN u.Username AS MostFollowedUser, numFollowers

f) Display first 5 users

MATCH (u:User)

RETURN u.Username AS Username, u.UserID AS UserID

ORDER BY u.Username ASC

LIMIT 5

5.CRUD Operations in Neo4j - Movie Recommendation Database: where we have nodes: Movie (MovieID, Title) and User (UserID, Username), and relationships: (LIKES) indicating a user likes a specific movie.

- Write a query to display all movies liked by a specific user with Username "John".
- Write a query to find all users who like the movie titled "Inception".
- Write a query to display all movies in ascending order by Title.
- Write a query to find users who like both "The Matrix" and "Inception".
- Write a query to find the most liked movie.
- Write a query to display the top 5 users who like the most movies.

// Create Users

CREATE

(u1:User {UserID: 1, Username: "John"}),

(u2:User {UserID: 2, Username: "Alice"}),

(u3:User {UserID: 3, Username: "Bob"}),

(u4:User {UserID: 4, Username: "Sara"}),



Date:

Course Code:23DS6PCNGD

Course:Next Gen Databases

USN No.:

```
(u5:User {UserID: 5, Username: "Mike"}),  
(u6:User {UserID: 6, Username: "Emma"}),  
(u7:User {UserID: 7, Username: "Liam"}),  
(u8:User {UserID: 8, Username: "Olivia"}),  
(u9:User {UserID: 9, Username: "James"}),  
(u10:User {UserID: 10, Username: "Sophia"});
```

// Create Movies

CREATE

```
(m1:Movie {MovieID: 101, Title: "Inception"}),  
(m2:Movie {MovieID: 102, Title: "The Matrix"}),  
(m3:Movie {MovieID: 103, Title: "Interstellar"}),  
(m4:Movie {MovieID: 104, Title: "Titanic"}),  
(m5:Movie {MovieID: 105, Title: "Avengers: Endgame"}),  
(m6:Movie {MovieID: 106, Title: "The Godfather"}),  
(m7:Movie {MovieID: 107, Title: "The Dark Knight"}),  
(m8:Movie {MovieID: 108, Title: "Pulp Fiction"}),  
(m9:Movie {MovieID: 109, Title: "Fight Club"}),  
(m10:Movie {MovieID: 110, Title: "Forrest Gump"});
```

// Create LIKES Relationships

CREATE

```
(u1)-[:LIKES]->(m1),  
(u1)-[:LIKES]->(m2),  
(u1)-[:LIKES]->(m3),  
(u2)-[:LIKES]->(m1),  
(u2)-[:LIKES]->(m4),  
(u3)-[:LIKES]->(m2),
```



Date:

Course Code:23DS6PCNGD

Course:Next Gen Databases

USN No.:

(u3)-[:LIKES]->(m5),
(u4)-[:LIKES]->(m3),
(u4)-[:LIKES]->(m6),
(u5)-[:LIKES]->(m1),
(u5)-[:LIKES]->(m2),
(u5)-[:LIKES]->(m7),
(u6)-[:LIKES]->(m1),
(u6)-[:LIKES]->(m8),
(u7)-[:LIKES]->(m9),
(u8)-[:LIKES]->(m10),
(u9)-[:LIKES]->(m2),
(u9)-[:LIKES]->(m1),
(u10)-[:LIKES]->(m3),
(u10)-[:LIKES]->(m1);

a) Query to display all movies liked by a specific user with Username "John".

```
MATCH (u:User {Username: "John"})-[:LIKES]->(m:Movie)
RETURN m.Title AS Movie;
```

b) Query to find all users who like the movie titled "Inception".

```
MATCH (u:User)-[:LIKES]->(m:Movie {Title: "Inception"})
RETURN u.Username AS User;
```

c) Query to display all movies in ascending order by Title.

```
MATCH (m:Movie)
RETURN m.Title AS Movie
ORDER BY m.Title ASC;
```

d) Query to find users who like both "The Matrix" and "Inception".



Date:

Course Code:23DS6PCNGD

Course:Next Gen Databases

USN No.:

```
MATCH (u:User)-[:LIKES]->(m1:Movie {Title: "The Matrix"}),  
      (u)-[:LIKES]->(m2:Movie {Title: "Inception"})  
RETURN u.Username AS User;
```

e) Query to find the most liked movie.

```
MATCH (m:Movie)<-[:LIKES]-(u:User)  
RETURN m.Title AS Movie, COUNT(u) AS Likes  
ORDER BY Likes DESC  
LIMIT 1;
```

f) Query to display the top 5 users who like the most movies.

```
MATCH (u:User)-[:LIKES]->(m:Movie)  
RETURN u.Username AS User, COUNT(m) AS MoviesLiked  
ORDER BY MoviesLiked DESC  
LIMIT 5;
```

Program 6:Redis Query Questions - Product Catalog Scenario

Create a product catalog database with products having attributes: (ProductID, Name, Category, Price)

- Insert products into Redis with keys like product:{ProductID} and values as JSON objects containing Name, Category, and Price.**
- Retrieve details of a specific product by ProductID.**
- Fetch all products belonging to a specific category (Electronics)**
- List Products in a Price Range (500 - 1000)**
- Update Product Price**
- Delete a Product**

a) Insert Products

- HSET product:101 Name "Laptop" Category "Electronics" Price 900
- HSET product:102 Name "Smartphone" Category "Electronics" Price 700
- HSET product:103 Name "Desk" Category "Furniture" Price 850



Date:

Course Code:23DS6PCNGD

Course:Next Gen Databases

USN No.:

- HSET product:104 Name "Tablet" Category "Electronics" Price 550

b)Retrieve Details of a Product by ProductID

- HGETALL product:102

c) Fetch All Products Belonging to a Category

- Redis hashes don't support filtering directly, so you must **manually maintain sets** as category indexes:
- SADD category:Electronics product:101 product:102 product:104
- SADD category:Furniture product:103
- Then retrieve members:
- SMEMBERS category:Electronics
- Then for each product ID:
- HGETALL product:101
- HGETALL product:102
- HGETALL product:104

d) List Products in a Price Range (500–1000)

- Use a **sorted set** with Price as score:
- ZADD price_index 900 product:101
- ZADD price_index 700 product:102
- ZADD price_index 850 product:103
- ZADD price_index 550 product:104
- Now query:
- ZRANGEBYSCORE price_index 500 1000
- Then get their details:
- HGETALL product:<ID>

e) Update Product Price

- HSET product:104 Price 600
- ZADD price_index 600 product:104

f) Delete a Product

- DEL product:103
- ZREM price_index product:103
- SREM category:Furniture product:103

Program 7:Consider an employee management system where each employee has attributes: (EmployeeID, Name, Department, Position, Salary)



Date:

Course Code:23DS6PCNGD

Course:Next Gen Databases

USN No.:

- a) **Insert Employee Details into Redis:** Insert employee records into Redis with keys like `employee:{EmployeeID}` and values as JSON objects containing Name, Department, Position, and Salary.
- b) **Retrieve All Employees in a Specific Department (e.g., "HR"):** Write a Redis query to fetch all employees belonging to a specific department, such as "HR."
- c) **List Employees with a Salary Above a Certain Amount (e.g., \$50,000):** Write a Redis query to list all employees whose salaries are above a certain threshold (e.g., \$50,000).

Update an Employee's Position: Write a Redis query to update the position of a specific employee identified by their EmployeeID

a) Insert Employee Details into Redis

HSET employee:101 Name "John Doe" Department "HR" Position "Manager" Salary 55000

HSET employee:102 Name "Jane Smith" Department "Engineering" Position "Developer" Salary 60000

HSET employee:103 Name "Alice Brown" Department "HR" Position "Recruiter" Salary 48000

Optional: maintain department index

SADD department:HR 101

SADD department:Engineering 102

Optional: maintain salary index using sorted set

ZADD employee_salaries 55000 101

ZADD employee_salaries 60000 102

ZADD employee_salaries 48000 103

b) Retrieve All Employees in a Specific Department (e.g., "HR")

SMEMBERS department:HR

returns: 101, 103



Date:

Course Code:23DS6PCNGD

Course:Next Gen Databases

USN No.:

Then get details:

HGETALL employee:101

HGETALL employee:103

c) List Employees with a Salary Above \$50,000

ZREVRANGEBYSCORE employee_salaries +inf 50000

returns:SMEMBERS department:HR

102, 101

Get details:

HGETALL employee:102

HGETALL employee:101

d) Update an Employee's Position

HSET employee:101 Position "Senior Manager"